📌 1. Definition of Singly Linked List

- **Singly linked lists** are a linear data structure
- Each element (node) contains:
  - **Data**
  - **Reference (pointer)** to the next node in the sequence

📌 2. Overview of Linked List Structure

- Linked lists consist of **nodes**
- Each node contains:
  - Data
  - Reference to the next node

📌 3. Purpose and Use Cases

- Used when a **dynamic data structure** is required
- Useful when:
  - Data size changes frequently

📌 4. Characteristics of a Singly Linked List

✓ Linear Data Structure

- Nodes are arranged in a sequence
- Each node points to the next node

✓ Dynamic Size

- Can **grow or shrink dynamically**
- Unlike arrays, no fixed size

✓ Node Structure

- Each node contains:
  - Data
  - Pointer to next node

📌 5. Why Use Singly Linked Lists?

✓ Advantages over Arrays

- Dynamic sizing
- Efficient insertion and deletion
- No need for contiguous memory

✓ Flexible Memory Usage

- Nodes can be stored anywhere in memory

✓ Applications

- Useful when:
  - Data size is not fixed
  - Frequent insertions/deletions

📌 6. Linked List Implementation (C)

✓ Overview

- Define a structure for node
- Use pointers (especially **head pointer**)

✓ Node Representation

- Each node has:

    - Data field
    - Pointer field

✓ Head Pointer

- Points to the **first node**

✓ Example Structure

```
struct Node {
   int data;        // Data stored in node
   struct Node* next; // Pointer to next node
};
```

📌 7. Basic Linked List Operations

✓ Insert Operations

- **insertFront** → Add node at beginning
- **insertEnd** → Add node at end

✓ Delete Operations

- **deleteFront** → Remove first node
- **deleteEnd** → Remove last node

✓ Search

- Find a specific value

✓ Demonstration

- Use code snippets for implementation

📌 8. Traversing a Linked List

```
struct node *temp = head;
```

```
printf("\nList elements are - \n");

while(temp != NULL) {
    printf("%d --->", temp->data);
    temp = temp->next;
}
```

📌 9. Insert at the Beginning

Steps:

- Allocate memory for new node
- Store data
- Point new node's next to head
- Update head

Code:

```
struct node *newNode;

newNode = malloc(sizeof(struct node));
newNode->data = 4;

newNode->next = head;
head = newNode;
```

📌 10. Insert at the Middle

Steps:

- Allocate memory
- Traverse to position
- Update pointers

Code:

```
struct node *newNode;
newNode = malloc(sizeof(struct node));
```

```
newNode->data = 4;

struct node *temp = head;

for(int i = 2; i < position; i++) {
    if(temp->next != NULL) {
        temp = temp->next;
    }
}

newNode->next = temp->next;
temp->next = newNode;
```

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *head = NULL;

struct Node* createNode(int value) {
    struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));
    if (newNode == NULL) {
        printf("Memory allocation failed\n");
        exit(1);
    }
    newNode->data = value;
    newNode->next = NULL;
    return newNode;
}

void insertBeginning(int value) {
    struct Node *newNode = createNode(value);
    newNode->next = head;
    head = newNode;
    printf("Node inserted at beginning\n");
}

void insertLast(int value) {
    struct Node *newNode = createNode(value);
```

```
    if (head == NULL) {
        head = newNode;
    } else {
        struct Node *temp = head;
        while (temp->next != NULL) {
            temp = temp->next;
        }
        temp->next = newNode;
    }
    printf("Node inserted at last\n");
}

void insertAtLocation(int value, int pos) {
    if (pos <= 1 || head == NULL) {
        insertBeginning(value);
        return;
    }

    struct Node *newNode = createNode(value);
    struct Node *temp = head;
    int i;

    for (i = 1; i < pos - 1 && temp->next != NULL; i++) {
        temp = temp->next;
    }

    newNode->next = temp->next;
    temp->next = newNode;
    printf("Node inserted at location %d\n", pos);
}

void deleteBeginning() {
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }

    struct Node *temp = head;
    head = head->next;
    free(temp);
    printf("Node deleted from beginning\n");
}

void deleteLast() {
```

```
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }

    if (head->next == NULL) {
        free(head);
        head = NULL;
        printf("Last node deleted\n");
        return;
    }

    struct Node *temp = head;
    while (temp->next->next != NULL) {
        temp = temp->next;
    }

    free(temp->next);
    temp->next = NULL;
    printf("Last node deleted\n");
}

void deleteAfterLocation(int pos) {
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }

    struct Node *temp = head;
    int i;

    for (i = 1; i < pos && temp != NULL; i++) {
        temp = temp->next;
    }

    if (temp == NULL || temp->next == NULL) {
        printf("Deletion not possible after location %d\n", pos);
        return;
    }

    struct Node *ptr = temp->next;
    temp->next = ptr->next;
    free(ptr);
    printf("Node deleted after location %d\n", pos);
```

```
}

void searchElement(int key) {
    struct Node *temp = head;
    int pos = 1;
    int found = 0;

    while (temp != NULL) {
        if (temp->data == key) {
            printf("Element %d found at position %d\n", key, pos);
            found = 1;
        }
        temp = temp->next;
        pos++;
    }

    if (!found) {
        printf("Element not found\n");
    }
}

void show() {
    struct Node *temp = head;
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }

    printf("Linked List: ");
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

int main() {
    int choice, value, pos;

    while (1) {
        printf("\nChoose one option from the following list ...\n");
        printf("==================================================\n");
        printf("1. Insert in beginning\n");
        printf("2. Insert at last\n");
```

```
printf("3. Insert at any random location\n");
printf("4. Delete from Beginning\n");
printf("5. Delete from last\n");
printf("6. Delete node after specified location\n");
printf("7. Search for an element\n");
printf("8. Show\n");
printf("9. Exit\n");

printf("Enter your choice? ");
scanf("%d", &choice);

switch (choice) {
    case 1:
        printf("Enter value: ");
        scanf("%d", &value);
        insertBeginning(value);
        break;

    case 2:
        printf("Enter value: ");
        scanf("%d", &value);
        insertLast(value);
        break;

    case 3:
        printf("Enter value: ");
        scanf("%d", &value);
        printf("Enter position: ");
        scanf("%d", &pos);
        insertAtLocation(value, pos);
        break;

    case 4:
        deleteBeginning();
        break;

    case 5:
        deleteLast();
        break;

    case 6:
        printf("Enter location after which node is to be deleted: ");
        scanf("%d", &pos);
        deleteAfterLocation(pos);
```

```
            break;

        case 7:
            printf("Enter element to search: ");
            scanf("%d", &value);
            searchElement(value);
            break;

        case 8:
            show();
            break;

        case 9:
            printf("Exiting...\n");
            exit(0);

        default:
            printf("Invalid choice\n");
    }
  }

  return 0;
}
```

Sample operations this code supports:

- Insert in beginning
- Insert at last
- Insert at any random location
- Delete from beginning
- Delete from last
- Delete node after specified location
- Search for an element
- Show list
- Exit

📌 Memory Management in Singly Linked Lists

✓ Dynamic Memory Allocation using `malloc` and `free`

- Nodes are dynamically allocated using **malloc()**
- Memory is deallocated using **free()** to prevent memory leaks

---

✓ Importance of Freeing Memory

- It is crucial to free allocated memory
- Helps to:
  - Avoid **memory leaks**
  - Ensure **efficient memory usage**

✓ Diagram Illustrating Memory Allocation

- Visual representation showing:
  - How memory is allocated for nodes
  - Nodes can be stored at **different locations in memory** (non-contiguous)

📌 Question

Explain how a singly linked list can be used to manage an employee database. Write a C program to store employee details using a linked list and display them.

📌 Solution

✓ Explanation:

A **singly linked list** can be used to manage an employee database where:

- Each node represents an **employee record**
- Each node contains:
  - Employee ID
  - Name

    - Pointer to the next employee

This allows:

- Dynamic addition/removal of employees
- Efficient memory usage

📌 C Program: Employee Database using Linked List

```
#include <stdio.h>
#include <stdlib.h>

struct Employee {
    int id;
    char name[50];
    struct Employee *next;
};

struct Employee *head = NULL;

// Create new employee
struct Employee* createEmployee(int id, char name[]) {
    struct Employee *newEmp = (struct Employee*)malloc(sizeof(struct Employee));
    newEmp->id = id;
    strcpy(newEmp->name, name);
    newEmp->next = NULL;
    return newEmp;
}

// Insert employee at end
void insertEmployee(int id, char name[]) {
    struct Employee *newEmp = createEmployee(id, name);

    if (head == NULL) {
        head = newEmp;
    } else {
        struct Employee *temp = head;
        while (temp->next != NULL) {
            temp = temp->next;
```

```
        }
        temp->next = newEmp;
    }
}

// Display employees
void displayEmployees() {
    struct Employee *temp = head;

    if (temp == NULL) {
        printf("No employee records found\n");
        return;
    }

    printf("\nEmployee Records:\n");
    while (temp != NULL) {
        printf("ID: %d, Name: %s\n", temp->id, temp->name);
        temp = temp->next;
    }
}

int main() {
    insertEmployee(101, "Alice");
    insertEmployee(102, "Bob");
    insertEmployee(103, "Charlie");

    displayEmployees();

    return 0;
}
```

📌 Output

```
Employee Records:
ID: 101, Name: Alice
ID: 102, Name: Bob
ID: 103, Name: Charlie
```

📌 Advantages of Using Singly Linked Lists

- ✓ **Dynamic size and memory utilization**
  - Size can grow or shrink as needed
  - No memory wastage
- ✓ **Efficient insertion and deletion**
  - No shifting of elements required
  - Faster than arrays in many cases
- ✓ **Versatility in data management**
  - Can be used in various applications
  - Flexible structure

📌 Time Complexity (Worst Case)

Operation	Time Complexity
Insert at the beginning	O(1)
Insert at the end	O(n)
Insert at a specific position	O(n)
Delete from the beginning	O(1)
Delete from the end	O(n)
Delete from a specific position	O(n)
Search	O(n)

📌 Common Mistakes and Pitfalls

- ❌ **Forgetting to update pointers**
  - Leads to broken links in list
- ❌ **Incorrect handling of edge cases**

    - Empty list
    - Single node list
- ❌ **Failure to free allocated memory**
    - Causes **memory leaks**